

作业 HW4 实验报告

姓名：庄子硕 学号：2350998 日期：2024 年 11 月 29 日

- # 实验报告格式要求按照模板（使用 Markdown 等也请保证报告内包含模板中的要素）
- # 对字体大小、缩进、颜色等不做强制要求（但尽量代码部分和文字内容有一定区分，可参考 vscode 配色）
- # 实验报告要求在文字简洁的同时将内容表示清楚
- # 报告内不要大段贴代码，尽量控制在 20 页以内

1. 涉及数据结构和相关背景

图和图的遍历

2. 实验内容

2.1 图的遍历

2.1.1 问题描述

本题给定一个无向图，用 dfs 和 bfs 找出图的所有连通分量。
所有顶点用 0 到 $n-1$ 表示，搜索时总是从编号最小的顶点出发。使用邻接矩阵存储，或者邻接表（使用邻接表时需要使用尾插法）。

2.1.2 基本要求

输入

第 1 行输入 2 个整数 n m ，分别表示顶点数和边数，空格分割
后面 m 行，每行输入边的两个顶点编号，空格分割

输出

第 1 行输出 dfs 的结果
第 2 行输出 bfs 的结果
连通子集输出格式为 $\{v_{11} \ v_{12} \ \dots\} \{v_{21} \ v_{22} \ \dots\} \dots$ 连通子集内元素之间用空格分割，子集之间无空格， $\{$ 和子集内第一个数字之间、 $\}$ 和子集内最后一个元素之间、子集之间均无空格

2.1.3 数据结构设计

- # 图、描述均可
- # 贴出数据结构定义代码（变量、结构体或类）并使用注释辅助说明
- # 例如

```

//邻接表
typedef struct ArcNode {
    int adjvex; //该弧所指向的顶点的位置
    struct ArcNode* nextarc; //指向下一条弧的指针
}ArcNode; //边结点类型

typedef struct VNode {
    VertexType data;
    int visited = 0;
    //顶点信息
    ArcNode* firstarc=nullptr; //指向第一条依附该顶点的指针
    ArcNode* lastarc = nullptr;
}VNode, AdjList[MAX_VERTEX_NUM];

typedef struct {
    AdjList vertices; //邻接表
    int vexnum, arcnum;
}ALGraph;

```

2.1.4 功能说明（函数、类）

```

int bfs(int src)
{
    if (gl.vertices[src].visited == 1)
        return 0;

    queue<int> q1;
    q1.push(src);
    buffer << src << ' ';
    gl.vertices[src].visited = 1;

    int count = 1;

    while (!q1.empty())
    {
        int cur = q1.front();
        q1.pop();

        ArcNode* curptr = gl.vertices[cur].firstarc;
        while (curptr != nullptr)
        {
            if (!gl.vertices[curptr->adjvex].visited) //因为是图，可能有环?
            {
                gl.vertices[curptr->adjvex].visited = 1;
                buffer << curptr->adjvex << ' ';
                q1.push(curptr->adjvex);
                count++;
            }
            curptr = curptr->nextarc;
        }
    }

    return count;
}

```

```

//遍历过的是点
int dfs(int src)
{
    if (gl.vertices[src].visited == 1)
        return 0;

    gl.vertices[src].visited = 1;
    int count = 0;
    count++;
    buffer << src << ' ';

    ArcNode* cur = gl.vertices[src].firstarc;
    while (cur != nullptr)
    {
        if (!(gl.vertices[cur->adjvex].visited))//因为是图，可能有环？
        {
            count+=dfs(cur->adjvex);
        }
        cur = cur->nextarc;
    }
    return count;
}

```

注释有错：保证无环 DAG

Bfs 输入开始遍历的节点下标，输出遍历到的节点数量。通过 visit 数组标记已经遍历的结点。

Dfs 同上

2.1.5 调试分析（遇到的问题 and 解决方法）

问题 1：bfs 正确，dfs 错误

解决方案 1：bfs 后没有清空 visit 数组

问题 2：输出末尾多一个空格

解决方案 2：输出到字符流中，编辑后输出

2.1.6 总结和体会

难点：输出控制

2.2 六度空间

2.2.1 问题描述

六度空间理论又称小世界理论。理论通俗地解释为：“你和世界上任何一个陌生人之间所间隔的人不会超过 6 个人，也就是说，最多通过五个人你就能够认识任何一个陌生人。”如图 1 所示。假如给你一个社交网络图，请你对每个节点计算符合“六度空间”理论的结点占结点总数的百分比。说明：由于浮点数精度不同导致结果有误差，请按 float 计算。

2.2.2 基本要求

输入

第 1 行给出两个正整数，分别表示社交网络图的结点数 N ($1 < N \leq 2000$ ，表示人数)、边数 M ($\leq 33 \times N$ ，表示社交关系数)。

随后的 M 行对应 M 条边，每行给出一对正整数，分别是该条边直接连通的两个结点的编号（节点从 1 到 N 编号）。

输出

对每个结点输出与该结点距离不超过 6 的结点数占结点总数的百分比，精确到小数点后 2 位。每个结节点输出一行，格式为“结点编号: (空格) 百分比%”。

2.2.3 数据结构设计

```
1
2
3
4
5
6
7
8     typedef struct ArcNode {
9         int adjvex; //该弧所指向的顶点的位置
10        struct ArcNode* nextarc; //指向下一条弧的指针
11    } ArcNode; //边结点类型
12
13     typedef struct VNode {
14         VertexType dist=-1;
15         //顶点信息
16         ArcNode* firstarc = nullptr; //指向第一条依附该顶点的指针
17         ArcNode* lastarc = nullptr;
18     } VNode, AdjList[MAX_VERTEX_NUM];
19
20     typedef struct {
21         AdjList vertices; //邻接表
22         int vexnum, arcnum;
23     } ALGraph;
24
25
26     ALGraph g1;
```

2.2.4 功能说明（函数、类）

```
for (int i = 1; i <= g1.vexnum; i++)
{
    int res = dfs(i, 0);

    cout << i << ": " << setprecision(2) << setiosflags(ios::fixed) << ((float)res / g1.vexnum) * 100 << "%" << endl;

    //清空visited
    for (int i = 1; i <= g1.vexnum; i++)
        g1.vertices[i].dist = -1;
}
```

```

//深度优先的搜索:当前节点、当前节点距离根节点的距离, 返回遍历的节点数
int dfs(int src,int depth)
{
    if (depth > 6)return 0;

    if (gl.vertices[src].dist <=depth)
        return 0;

    int count = 0;

    if (gl.vertices[src].dist == -1)count++;
    gl.vertices[src].dist = depth;

    ArcNode* cur = gl.vertices[src].firstarc;
    while (cur != nullptr)
    {
        if (gl.vertices[cur->adjvex].dist > depth+1)//因为是图, 可能有环?
        {
            count += dfs(cur->adjvex, depth+1);
        }
        cur = cur->nextarc;
    }
    return count;
}

```

Dfs 输入搜索的开始下标, 和从当前下标开始的最大搜索深度, 返回搜索到的元素个数。

从每个节点开始 dfs, 计算六度以内的比例, 并重置 visit 数组

2.2.5 调试分析（遇到的问题 and 解决方法）

问题 1: 第一次正确, 后续错误

解决方案 1: bfs 后没有清空 visit 数组

2.2.6 总结和体会

需要有深度限制的 dfs（迭代加深搜索？）

2.3 村村通

2.3.1 问题描述

N 个村庄, 从 1 到 N 编号, 现在请你修建一些路使得任何两个村庄都彼此连通。我们称两个村庄 A 和 B 是连通的, 当且仅当在 A 和 B 之间存在一条路, 或者存在一个村庄 C, 使得 A 和 C 之间有一条路, 并且 C 和 B 是连通的。

已知在一些村庄之间已经有了一些路, 您的工作是再兴建一些路, 使得所有的村庄都是连通的, 并且新建的路的长度是最小的。

2.3.2 基本要求

输入

第一行包含一个整数 n ($3 \leq n \leq 100$), 表示村庄数目。

接下来 n 行,每行 n 个非负整数,表示村庄 i 和村庄 j 之间的距离。距离值在 $[1,1000]$ 之间。
接着是一个整数 m ,后面给出 m 行,每行包含两个整数 $a,b,(1 \leq a < b)$,表示在村庄 a 和 b 之间已经修建了路。

输出

输出一行,仅有一个整数,表示为使所有的村庄连通,要新建公路的长度的最小值。

2.3.3 数据结构设计

```
//采用Prim算法,取第一个点开始
typedef int VRType;

struct {
    VertexType adjvex;
    VRType lowcost;
} closedge[MAX_VERTEX_NUM]; //辅助表

//定义一个邻接矩阵
typedef struct {
    int arcs[MAX_VERTEX_NUM][MAX_VERTEX_NUM];
    VertexType vexs[MAX_VERTEX_NUM];
    int vexnum, arcnum;
} MGraph;
MGraph G1;
```

2.3.4 功能说明 (函数、类)

```

//u为选定的顶点
void MiniSpanTree_PRIM(const MGraph& G, VertexType u)
{
    int count = 0;
    int k, j, i, minCost;
    k = LocateVex(u); //表示从节点自身信息到其索引的转换
    //初始化辅助表
    for (j = 0; j < G.vexnum; ++j)
        if (j != k)
        {
            closedge[j].adjvex = u;
            closedge[j].lowcost = G.arcs[k][j];
        }

    closedge[k].adjvex = u;
    closedge[k].lowcost = -1;

    //遍历 边数-1次
    for (i = 0; i < G.vexnum-1; ++i)
    { // k=minimum(closedge);
        minCost = 10000000;
        for (j = 0; j < G.vexnum; ++j)
        {
            if (closedge[j].lowcost < minCost &&
                closedge[j].lowcost != -1)
            {
                minCost = closedge[j].lowcost;
                k = j;
            }
        }

        //printf("(%d,%d)\n", closedge[k].adjvex, G.vexs[k]);
        count += G.l.arcs[k][LocateVex(closedge[k].adjvex)];
        //把选定的点加入点集
        closedge[k].lowcost = -1;
        //更新closedge辅助表
        for (j = 0; j < G.vexnum; ++j)
            if (G.arcs[k][j] < closedge[j].lowcost)
            {
                closedge[j].adjvex = G.vexs[k];
                closedge[j].lowcost = G.arcs[k][j];
            }
    }

    cout << count<<endl;
}

```

预定义已经相连的村权为 0，然后调用 Prim 算法生成最小生成树

2.3.5 调试分析（遇到的问题解决方法）

问题 1：标志位 0 与距离 0 冲突

解决方案 1: 标志位设置为-1

2.3.6 总结和体会

多使用测试语句

2.4 给定条件下构造矩阵

2.4.1 问题描述

给你一个正整数 k ，同时给你：

一个大小为 n 的二维整数数组 `rowConditions`，其中 `rowConditions[i]=[abovei,belowi]` 和一个大小为 m 的二维整数数组 `colConditions`，其中 `colConditions[i]=[lefti,righti]`。两个数组里的整数都是 1 到 k 之间的数字。

你需要构造一个 $k \times k$ 的矩阵，1 到 k 每个数字需要恰好出现一次。剩余的数字都是 0。矩阵还需要满足以下条件：对于所有 0 到 $n-1$ 之间的下标 i ，数字 `abovei` 所在的行必须在数字 `belowi` 所在行的上面。对于所有 0 到 $m-1$ 之间的下标 i ，数字 `lefti` 所在的列必须在数字 `righti` 所在列的左边。

返回满足上述要求的矩阵，题目保证若矩阵存在则一定唯一；如果不存在答案，返回一个空的矩阵。

2.4.2 基本要求

输入：

第一行包含 3 个整数 k 、 n 和 m

接下来 n 行，每行两个整数 `abovei`、`belowi`，描述 `rowConditions` 数组

接下来 m 行，每行两个整数 `lefti`、`righti`，描述 `colConditions` 数组

输出：

如果可以构造矩阵，打印矩阵；否则输出 -1

矩阵中每行元素使用空格分隔

2.4.3 数据结构设计

```

    typedef struct ArcNode {
        int adjvex; //该弧所指向的顶点的位置, 链接表中对应元素的的下标
        struct ArcNode* nextarc; //指向下一条弧的指针
    } ArcNode; //边结点类型

    typedef struct VNode {
        VertexType data;
        int sta=0; //表示是否被访问过/被访问中, 1已经访问, 0未访问, -1正在访问
        //顶点信息
        ArcNode* firstarc = nullptr; //指向第一条依附该顶点的指针
        ArcNode* lastarc = nullptr;

        void push(ArcNode* newptr)
        {
            if (firstarc == nullptr)
            {
                firstarc = newptr;
                lastarc = firstarc;
            }
            else
            {
                ArcNode* mid = lastarc;
                lastarc = newptr;
                mid->nextarc = lastarc;
            }
        }
    } VNode, AdjList[MAX_VERTEX_NUM];

    typedef struct {
        AdjList vertices; //邻接表
        int vexnum, arcnum;
    } ALGraph;

```

邻接表

2.4.4 功能说明（函数、类）

```

//从下标为i1的节点开始dfs打印拓扑排序
//从拓扑排序小端输出到大端
bool dfs(ALGraph& g1, int i1)
{
    g1.vertices[i1].sta = -1;

    ArcNode* cur_arc = g1.vertices[i1].firstarc;
    while (cur_arc != nullptr)
    {
        int state = g1.vertices[cur_arc->adjvex].sta;
        if (state == -1) return false;
        else if (state == 0)
            if (!dfs(g1, cur_arc->adjvex)) return false;
        else;

        cur_arc = cur_arc->nextarc;
    }

    g1.vertices[i1].sta = 1;
    // cout << g1.vertices[i1].data; //打印数据
    s1.push(g1.vertices[i1].data);
    return true;
}

bool topo_sort(ALGraph& g1)
{
    for (int i = 0; i < g1.vexnum; i++)
    {
        if (g1.vertices[i].sta == 0)
            if (!dfs(g1, i)) return false;
    }

    return true;
}

```

拓扑排序：对邻接表中节点逐个作 dfs：每次 dfs 将所有小于该节点的点全部遍历。

2.4.5 调试分析（遇到的问题和解决方法）

问题 1：已知横纵拓扑排序，如何求得矩阵

解决方案 1：对于一个 i, j 位置，如果 $col[i] = row[j]$ ，说明此位置为 $col[i]$ 。从而不需要生成矩阵，

只需要嵌套遍历 1-n 两次就可以。

2.4.6 总结和体会

多使用测试语句

2.5 必修课

2.5.1 问题描述

某校的计算机系有 n 门必修课程。学生需要修完所有必修课程才能毕业。

每门课程都需要一定的学时去完成。有些课程有前置课程，需要先修完它们才能修这些课程；而其他课程没有。不同于大多数学校，学生可以在任何时候进行选课，且同时选课的数量没有限制。

现在校方想要知道：

1. 从入学开始，每门课程最早可能完成的时间（单位：学时）；
2. 对每一门课程，若将该课程的学时增加 1，是否会延长入学到毕业的最短时间。

2.5.2 基本要求

输入：

第一行，一个正整数 n ，代表课程的数量。

接下来 n 行，每行若干个整数：

- 第一个整数为 t_i ，表示修完该课程所需的学时。
- 第二个整数为 c_i ，表示该课程的前置课程数量。
- 接下来 c_i 个互不相同的整数，表示该课程的前置课程的编号。

该校保证，每名入学的学生，一定能够在有限的时间内毕业。

输出：

输出共 n 行，第 i 行包含两个整数：

- 第一个整数表示编号为 i 的课程最早可能完成的时间。
- 第二个整数表示，如果将该课程的学时增加 1，入学到毕业的最短时间是否会增加。如果会增加则输出 11，否则输出 00。

每行的两个整数以一个空格隔开。

2.5.3 数据结构设计

```

typedef struct ArcNode {
    // int weight; //边的权值
    int adjvex; //该弧所指向的顶点的位置, 链接表中对应元素的下的标
    struct ArcNode* nextarc; //指向下一条弧的指针
} ArcNode; //边结点类型

typedef struct VNode {
    VertexType data;
    int cost;
    int sta = 0; //表示是否被访问过/被访问中, 1已经访问, 0未访问, -1正在访问
    //顶点信息
    ArcNode* firstarc = nullptr; //指向第一条依附该顶点的指针
    ArcNode* lastarc = nullptr;

    void push(ArcNode* newptr)
    {
        if (firstarc == nullptr)
        {
            firstarc = newptr;
            lastarc = firstarc;
        }
        else
        {
            ArcNode* mid = lastarc;
            lastarc = newptr;
            mid->nextarc = lastarc;
        }
    }
} VNode, AdjList[MAX_VERTEX_NUM];

typedef struct {
    AdjList vertices; //邻接表
    int vexnum, arcnum;
} ALGraph;

ALGraph G1;

```

邻接表

2.5.4 功能说明（函数、类）

根据顶点信息，查找索引

```

int LocateVex(const ALGraph& g1, VertexType d1)
{
    for (int i = 0; i < g1.vexnum; i++)
        if (g1.vertices[i].data == d1)
            return i;
    return g1.vexnum;
}

```

深度优先搜索，计算每个顶点的“总消耗”。

```

int dfs(ALGraph& g1, int* total_consumes, int il)

```

```

{
    gl.vertices[i1].sta = -1;
    int count = 0;
    ArcNode* cur_arc = gl.vertices[i1].firstarc;
    while (cur_arc != nullptr)
    {
        int state = gl.vertices[cur_arc->adjvex].sta;
        if (state == -1)return -1;
        else if (state == 0)
        {
            int mid=dfs(gl, total_consumes, cur_arc->adjvex);
            if (mid == -1)return -1;
            if (mid > count)
                count = mid;
        }
        else
        {
            if (total_consumes[cur_arc->adjvex] > count)
                count = total_consumes[cur_arc->adjvex];
            cur_arc = cur_arc->nextarc;
        }
    }
    gl.vertices[i1].sta = 1;
    total_consumes[i1] = count + gl.vertices[i1].cost;
    return total_consumes[i1];
}

```

拓扑排序，检查图中是否存在环。复杂度 $O(V+E)$

```

bool topo_sort(ALGraph& gl)
{
    for (int i = 0; i < gl.vexnum; i++)
    {
        if (gl.vertices[i].sta == 0)
            if (dfs(gl, total_consume, i)==-1)return false;
    }
    return true;
}

```

2.5.5 调试分析（遇到的问题 and 解决方法）

问题 1：答案错误，推断没有发现问题

解决方案 1：增加测试语句，打印最长子树

问题 2：不知道如何判断是否成环

解决方案 2：每个顶点分配一个状态位，-1 表示正在访问

2.5.6 总结和体会

多加测试语句；先写一个能跑的，再简化复杂度、STL；

2.6 小马吃草

2.6.1 问题描述

假设无向图 G 上有 N 个点和 M 条边，点编号为 1 到 N ，第 i 条边长度为 w_i ，其中 H 个点上有可以食用的牧草。另外有 R 匹小马，第 j 匹小马位于点 $start_j$ ，需要先前往任意一个有牧草的点进食牧草，然后前往点 end_j ，请你计算每一匹小马需要走过的最短距离。

2.6.2 基本要求

第一行两个整数 N 、 M ，分别表示点和边的数量

接下来 M 行，第 i 行包含三个整数 x_i, y_i, w_i ，表示从点 x_i 到点 y_i 有一条长度为 w_i 的边，保证 $x_i \neq y_i$

接下来一行有两个整数 H 和 R ，分别表示有牧草的点数量和小马的数量

接下来一行包含 H 个整数，为 H 个有牧草的点编号

接下来 R 行，第 j 行包含两个整数 $start_j$ 和 end_j ，表示第 j 匹小马起始位置和终点位置

题目保证两个点之间一定是连通的，并且至少有一个点上有牧草

2.6.3 数据结构设计

```
✓ typedef struct {  
    int arcs[MAX_VERTEX_NUM][MAX_VERTEX_NUM];  
    int vexnum, arcnum;  
} MGraph;  
MGraph G1;
```

邻接矩阵

2.6.4 功能说明（函数、类）

1. LocateVex 函数：

功能：将顶点编号转换为邻接矩阵中的下标。

参数：顶点编号 vex 。

返回值：顶点在邻接矩阵中的下标。

//输入编号，返回下标

```
inline int LocateVex(int vex)  
{  
    return vex - 1;  
}
```

2. Floyd 函数：

功能：使用弗洛伊德算法计算图中所有顶点之间的最短路径。

参数：顶点数量 vexnum。

返回值：无，直接修改全局变量 G1.arcs。

```
void Floyd(int vexnum)
{
    for (int k = 0; k < vexnum; k++)
    {
        for (int i = 0; i < vexnum; i++)
            for (int j = 0; j < vexnum; j++)
            {
                if (G1.arcs[i][k] != -1 && G1.arcs[k][j] != -1)
                    if (G1.arcs[i][j] == -1)
                        G1.arcs[i][j] = (G1.arcs[i][k]) + (G1.arcs[k][j]);
                    else
                        G1.arcs[i][j] = min((G1.arcs[i][j]), (G1.arcs[i][k]) +
(G1.arcs[k][j]));
            }
    }
}
```

3. main 函数：

功能：读取输入数据，构建图，调用弗洛伊德算法计算最短路径，并处理马吃草的问题。

详细步骤：

初始化图的邻接矩阵。

读取图的顶点数和边数，构建图。

读取草的位置，标记草的位置。

调用 Floyd 函数计算最短路径。

处理马吃草的问题，输出每匹马吃草的最短路径。

```
int grass[MAX_VERTEX_NUM];
//点的编号从1开始
int main()
{
    // -1表示正无穷
    memset(grass, 0, sizeof(int) * MAX_VERTEX_NUM);
    memset(G1.arcs, -1, sizeof(int) * MAX_VERTEX_NUM * MAX_VERTEX_NUM);

    cin >> G1.vexnum >> G1.arcnum;

    for (int i = 0; i < G1.arcnum; i++) {
        int src_name, dst_name, weight;
        cin >> src_name >> dst_name >> weight;

        if (G1.arcs[src_name - 1][dst_name - 1] == -1 || G1.arcs[src_name -
1][dst_name - 1] > weight) {
```

```

        G1.arcs[src_name - 1][dst_name - 1] = weight;
    }
    if (G1.arcs[dst_name - 1][src_name - 1] == -1 || G1.arcs[dst_name -
1][src_name - 1] > weight) {
        G1.arcs[dst_name - 1][src_name - 1] = weight;
    }
}

for (int i = 0; i < G1.vexnum; i++)
{
    G1.arcs[i][i] = 0;
}

int G, H;//草数和马数
cin >> G >> H;

for (int i = 0; i < G; i++)
{
    int grass_pos;
    cin >> grass_pos;
    grass[(grass_pos-1)] = 1;
}

Floyd(G1.vexnum);

for (int i = 0; i < H; i++)
{
    int src, dst;
    cin >> src >> dst;
    int min_cost = 1000000;//一个大数
    for (int j = 0; j < G1.vexnum; j++)
    {
        if (!grass[j])continue;
        if (G1.arcs[(src-1)][j] != -1 && G1.arcs[j][(dst-1)] != -1)
            min_cost = min(min_cost, (G1.arcs[(src-1)][j]) +
(G1.arcs[j][(dst-1)]));
    }
    cout << min_cost << endl;
}

return 0;
}

```

2.6.5 调试分析（遇到的问题 and 解决方法）

问题 1: TLE

解决方案 1: 把函数写成语句, 开大数组

问题 2: 如何判断无穷大

解决方案 2: 设为 -1; 每当遇到非 -1 才尝试更新, 如果原结点是 -1, 则一定更新; 不然, 则更新为最小值

问题 3: 输入处理错误: 在初始版本中, 没有考虑到两个顶点之间可能有多条边的情况, 导致输入处理错误

解决方案 3: 修改输入处理部分, 确保在读取每条边时, 只保留最短的边

问题 4: 视为有向图无解

解决方案 4: 题目要求无向图

2.6.6 总结和体会

算法选择: 弗洛伊德算法适用于计算图中所有顶点对之间的最短路径, 虽然时间复杂度较高 ($O(V^3)$), 但对于顶点数较少的图 (如本题中的 1200 个顶点), 仍然是一个可行的选择。

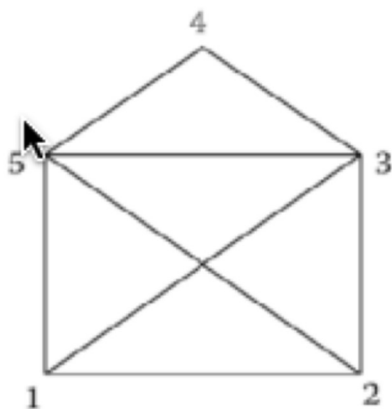
输入处理: 在处理输入时, 需要特别注意输入数据的格式和边界条件。本题中, 两个顶点之间可能有多条边, 需要选择最短的边进行处理。

调试技巧: 在调试过程中, 逐步检查每个步骤的输出, 确保每一步的计算结果都是正确的。特别是在处理图的邻接矩阵时, 需要仔细检查每个顶点的自环和无穷大的处理。

2.7 一笔画问题

2.7.1 问题描述

请你写一个程序, 从下图所示房子的左下角 (数字 1) 开始, 按照节点递增顺序, 输出所有可以一笔画完的顶点顺序 (欧拉路径), 要求所有的边恰好都只画一次。例如, 123153452 就是其中的一条路径。



2.7.2 基本要求

见上

2.7.3 数据结构设计

```

typedef struct {
    int arcs[MAX_VERTEX_NUM][MAX_VERTEX_NUM];
    int vexnum, arcnum;
} MGraph;

MGraph G1;

int visit[MAX_VERTEX_NUM][MAX_VERTEX_NUM] = { 0 };

```

邻接矩阵

2.7.4 功能说明（函数、类）

LocateVex 函数：

功能：将顶点编号转换为邻接矩阵中的下标。

参数：顶点编号 vex。

返回值：顶点在邻接矩阵中的下标。

```

inline int LocateVex(int vex)
{
    return vex - 1;
}

```

dfs 函数：

功能：深度优先搜索（DFS）遍历所有可能的路径，寻找欧拉回路。

参数：

route_depth：当前路径的深度。

seq：存储当前路径的数组。

seq_length：当前路径的长度。

point：当前顶点的编号。

返回值：无，直接输出找到的欧拉回路。

```

void dfs(int route_depth, int* seq, int seq_length, int point)
{
    if (route_depth >= 8)
    {
        for (int i = 0; i < 9; i++)
            cout << seq[i];
        cout << endl;
    }

    for (int i = 1; i <= 5; i++)
    {
        if (i == point) continue;
        if
(!visit[LocateVex(point)][LocateVex(i)] && G1.arcs[LocateVex(point)][LocateVex(i)])
        {

```

```

        seq[seq_length+1] = i;
        visit[LocateVex(point)][LocateVex(i)] = 1;
        visit[LocateVex(i)][LocateVex(point)] = 1;
        dfs(route_depth + 1, seq, seq_length + 1, i);
        visit[LocateVex(point)][LocateVex(i)] = 0;
        visit[LocateVex(i)][LocateVex(point)] = 0;
    }
}
}

```

main 函数:

功能: 初始化图的邻接矩阵, 调用 dfs 函数寻找欧拉回路。

详细步骤:

初始化图的邻接矩阵, 设置顶点之间的边。

设置特定顶点之间的边为 0, 表示这些边不存在。

调用 dfs 函数, 从顶点 1 开始寻找欧拉回路。

```

int main()
{
    for (int i = 0; i < 5; i++)
        for (int j = 0; j < 5; j++)
            Gl.arcs[i][j] = 1;
    for (int i = 0; i < 5; i++)
        Gl.arcs[i][i] = 0;
    Gl.arcs[LocateVex(1)][LocateVex(4)] = 0;
    Gl.arcs[LocateVex(4)][LocateVex(1)] = 0;
    Gl.arcs[LocateVex(2)][LocateVex(4)] = 0;
    Gl.arcs[LocateVex(4)][LocateVex(2)] = 0;
    int seq[10];
    seq[0] = 1;
    dfs(0, seq, 0, 1);
    return 0;
}

```

2.7.5 调试分析（遇到的问题 and 解决方法）

问题 1: 遍历到已经遍历过的边

解决方案 1: 设置错误, 应当设置遍历过的边数组, 而非点数组; 由于是无向图, 应当同时更新 $[i][j]$ 和 $[j][i]$

问题 2: 初始化错误

解决方案 2: 注意到编号和下标的区别

问题 3: 序列少一位

解决方案 3: 8 边 9 点而非 8 点, 打印循环少了 1

问题 4: 访问矩阵未正确重置:

解决方案 4: 在递归调用 dfs 后, 正确重置访问矩阵 visit。

2.7.6 总结和体会

算法选择:本实验采用深度优先搜索 (DFS) 遍历所有可能的路径, 寻找欧拉回路。虽然这种方法直观易懂, 但时间复杂度较高, 不适合处理大规模图。具体复杂度为 $O(\theta!)$, θ 是最大度数。

调试技巧:在调试过程中, 逐步检查每个步骤的输出, 确保每一步的计算结果都是正确的。特别是在处理图的邻接矩阵和访问矩阵时, 需要仔细检查每个顶点的状态和边的状态。

代码可读性:在编写代码时, 保持代码的可读性和模块化, 有助于后续的调试和维护。特别是在处理复杂逻辑时, 清晰的注释和函数封装可以大大提高代码的可维护性。

3. 实验总结

3.1 实验目的与目标

本次实验的主要目的是通过编程实现和分析一些图论算法, 包括图的遍历、最短路径、图的连通性问题以及最小生成树等问题, 旨在帮助我们深入理解和应用数据结构与算法, 特别是在图的建模、搜索与优化方面。具体任务包括:

1. 图的遍历: 使用 DFS 和 BFS 找出无向图的连通分量。
2. 六度空间: 基于小世界理论, 计算社交网络中的节点在六度空间内的比例。
3. 村村通: 通过图的最小生成树 (MST) 算法, 构建连接所有村庄的最小公路网络。
4. 必修课: 在带有前置条件的课程结构中, 计算每门课程的最早完成时间, 并判断修改课程学时对毕业时间的影响。
5. 小马吃草: 在图中计算每匹小马的最短路径, 满足去吃草再去终点的条件。
6. 一笔画问题: 通过深度优先搜索 (DFS) 找到欧拉路径。

3.2 算法实现与思路

1. 图的遍历 (DFS 与 BFS)
 - DFS 与 BFS 的应用: 在图的遍历中, DFS 与 BFS 是最基础的图遍历算法。DFS 通过递归的方式实现深度优先的搜索, 而 BFS 则是层级遍历。我们在解决图的连通分量问题时, DFS 的递归特性能够帮助我们快速找到所有的连通组件; BFS 则更加直观, 利用队列实现逐层遍历, 确保找到最短路径。
 - 遇到的挑战与解决方法: 主要问题是在进行 DFS 遍历时, 若未清空 visit 数组, 会导致后续遍历时产生错误, 特别是在多次连通分量的计算中。解决方法是每次开始新一轮 DFS 遍历时, 要重新初始化访问标记数组。
2. 六度空间问题
 - 深度限制 DFS: 由于六度空间问题要求计算从每个节点出发, 六度内的节点数量, 我们使用了带有深度限制的 DFS, 限制搜索的最大深度为 6。每次 DFS 都要统计节点在该深度内的数量, 并返回相应的结果。
 - 遇到的挑战与解决方法: 一个常见问题是 DFS 在计算深度时容易超出限制, 导致计算出错。解决方案是在 DFS 时严格控制深度, 并确保递归调用中传递正确的深度信息。
3. 村村通问题 (最小生成树)
 - Prim 算法: 解决村村通问题时, 我们采用了 Prim 算法来构建最小生成树。该算法通过从任意一个顶点开始, 逐渐扩展到与已选顶点相连的最短边, 直至所有顶点都被连接起来。我们需要特别注意已经存在路的情况, 在这些路的边权值上设置为零, 避免被重复计算。
 - 遇到的挑战与解决方法: 一个问题是在使用 Prim 算法时, 标志位的设置需要谨慎处理, 特别是在标记顶点是否已加入生成树时, 避免与边权的值产生冲突。解决方法是将标志位设置为 -1, 避免与 0 产生混淆。
4. 必修课问题

- 拓扑排序与 DFS：对于带有前置课程约束的课程安排问题，我们使用了拓扑排序来处理课程间的依赖关系。DFS 用于计算每个课程的最早完成时间，并且能够根据课程的修改判断是否会影响毕业的最短时间。
- 遇到的挑战与解决方法：在处理图的环检测时，我们需要确保图中没有环，这可以通过 DFS 中递归标记的状态来实现。若发现环，即可终止该路径的计算。

5. 小马吃草问题

- 最短路径与多源问题：为了计算每匹小马的最短路径，我们采用了 Floyd 算法，预先计算图中每对点之间的最短路径。在实际查询时，通过 Floyd 计算结果快速得到每匹小马从起点到终点的最短路径。
- 遇到的挑战与解决方法：一个挑战是如何处理图中的多条边以及“无穷大”表示的问题。通过设定邻接矩阵中不可达点的初始值为-1，确保每次计算时对无效路径的判断。

6. 一笔画问题（欧拉路径）

- 深度优先搜索：在欧拉路径问题中，我们使用 DFS 来递归搜索图中的所有路径，确保每条边恰好访问一次。由于是无向图，我们需要在遍历时同时更新两个方向的边状态。
- 遇到的挑战与解决方法：问题的关键是如何正确管理已访问边和节点。我们采用了一个 visit 矩阵来标记边是否已被访问过，同时确保每次递归时，能回退到上一个状态，避免重复访问边。

3.3 实验中的难点与体会

1. 图的建模：图的存储方式是算法实现的基础。在实验中，我们分别使用了邻接矩阵、邻接表等方式存储图数据，不同的存储结构适用于不同类型的图问题。例如，对于稠密图，邻接矩阵可以提高查询效率；对于稀疏图，邻接表则更加高效。在选择存储结构时，需要根据图的特点来优化空间和时间复杂度。
2. 算法优化：许多图算法，尤其是最短路径算法，如 Floyd 算法，在大规模图中可能会遇到性能瓶颈。实验中，我们使用了动态编程、优先队列等技术来优化算法，提高计算效率。同时，调试过程中也发现了算法边界条件处理不当等问题，必须通过细致的测试来验证每个边界条件。
3. 深度优先搜索的灵活性：在处理不同问题时，DFS 算法表现出了极大的灵活性，尤其是在连通性、路径搜索、拓扑排序等问题中都得到了应用。通过递归和回溯，我们能够有效地探索所有可能的路径，并根据需求进行剪枝和状态管理。
4. 调试与验证：本次实验过程中遇到了许多调试问题，比如输出格式错误、数组越界、路径计算错误等。每个问题的解决都需要耐心细致地分析代码逻辑，并通过增加测试语句、逐步检查每个计算步骤来定位问题。
5. 浮动问题与精度：在六度空间问题中，由于浮点数的精度问题，最终结果的百分比计算可能会出现细微的误差。为了保证输出的准确性，我们确保结果保留两位小数，并通过有效的输入输出控制避免误差的传播。

3.4 总结

通过本次实验，我深刻理解了图论中的几种经典算法（DFS、BFS、Floyd、Prim 等）的应用场景及其实现技巧。实验不仅提高了我对图数据结构的理解，也让我掌握了图算法的实现和优化技巧。在实践中，调试能力和问题分析能力变得尤为重要，通过逐步验证、逐层调试，最终得到了正确的结果。同时，实验中还提升了我的代码组织和算法优化能力，对于复杂问题的分解和实现提供了宝贵的经验。